

Section 89 Editorial — Polynomial Evaluation And Subset Transforms

A. Multipoint Evaluation

How to find it

Organize all moduli $x-X_i$ in a product tree and propagate remainders.

Correctness

Remainder transitivity makes each child remainder congruent to the original polynomial modulo its subtree product; at a leaf its constant is exactly the requested value.

Complexity

$O(M(n+q) \log q)$ time and $O((n+q) \log q)$ aggregate polynomial storage.

At each node store $P \bmod \text{product}[\text{node}]$; since a child product divides its parent product, reducing the parent's remainder again is equivalent to reducing the original polynomial. Repeated points are legal and simply create equal linear factors at different leaves.

C++

```
#include <bits/stdc++.h>
using namespace std; using ll=long long;
const int MOD=998244353,G=3;
int power(int a,int e){ll r=1;for(;e;e>>=1,a=(ll)a*a%MOD)if(e&1)r=r*a%MOD;return r;}
void ntt(vector<int>&a,bool inv){int n=a.size();for(int i=1,j=0;i<n;i++){int b=n>>1;for(;j&b;b>>=1)
vector<int> conv(vector<int>a,vector<int>b,int need=-1){if(a.empty()||b.empty())return{};int total=
vector<int> inverse(const vector<int>&a,int n){vector<int>r(1,power(a[0],MOD-2));for(int sz=2;sz/2<
vector<int> derivative(const vector<int>&a){vector<int>d(max(0,(int)a.size()-1));for(int i=1;i<(int
vector<int> integrate(const vector<int>&a){vector<int>r(a.size()+1),iv(a.size()+1);if(a.size())iv[1
vector<int> logarithm(const vector<int>&a,int n){auto r=integrate(conv(derivative(a),inverse(a,n),n
vector<int> exponential(const vector<int>&a,int n){vector<int>r(1,1);for(int sz=2;sz/2<n;sz<=1){in
vector<int> square_root(const vector<int>&a,int n){vector<int>r(1,1);int half=(MOD+1)/2;for(int sz=
void trim(vector<int>&a){while(!a.empty()&&a.back()==0)a.pop_back();}
vector<int> poly_mod(vector<int>a,vector<int>b){trim(a);trim(b);if(a.size()<b.size())return a;vector
struct ProductTree{int n,size;vector<int>x;vector<vector<int>>t;ProductTree(vector<int>xs):n(xs.size
```

```

vector<int> eval(const vector<int>&a){vector<int>ans(n);function<void(int,vector<int>)>go=[&](int t,vector<int> interp(const vector<int>&y){auto val=eval(derivative(t[1]));vector<vector<int>>w(2*size_t)&ans){void fwt(vector<int>&a,string kind,bool inv=false){int n=a.size();for(int bit=1;bit<n;bit<=1)for(int i=0;i<n;i+=bit)vector<int> subset_conv(const vector<int>&f,const vector<int>&g,int k){int n=1<<k;vector<vector<int>>w(2*size_t)&ans){void print(const vector<int>&a){for(int i=0;i<(int)a.size();i++)cout<<a[i]<<" \n"[i+1==(int)a.size()];}int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;cin>>n>>q;vector<int>a(n),x(q);for(

```

Python

```

import sys
MOD=998244353
ROOT=3
def ntt(a,invert):
    n=len(a);j=0
    for i in range(1,n):
        bit=n>>1
        while j&bit:j^=bit;bit>>=1
        j^=bit
        if i<j:a[i],a[j]=a[j],a[i]
    length=2
    while length<=n:
        wlen=pow(ROOT,(MOD-1)//length,MOD)
        if invert:wlen=pow(wlen,MOD-2,MOD)
        half=length>>1
        for i in range(0,n,length):
            w=1
            for j in range(i,i+half):
                u=a[j];v=a[j+half]*w%MOD
                a[j]=(u+v)%MOD;a[j+half]=(u-v)%MOD;w=w*wlen%MOD
            length<=1
    if invert:
        z=pow(n,MOD-2,MOD)
        for i in range(n):a[i]=a[i]*z%MOD
def conv(a,b,need=None):
    if not a or not b:return []
    total=len(a)+len(b)-1
    if min(len(a),len(b))<32:
        c=[0]*total
        for i,x in enumerate(a):
            for j,y in enumerate(b):c[i+j]=(c[i+j]+x*y)%MOD
    else:
        size=1
        while size<total:size<=1
        x=a+[0]*(size-len(a));y=b+[0]*(size-len(b));ntt(x,False);ntt(y,False)
        for i in range(size):x[i]=x[i]*y[i]%MOD
        ntt(x,True);c=x[:total]
    return c if need is None else c[:need]
def fps_inv(a,n):
    g=[pow(a[0],MOD-2,MOD)];size=1

```

```

while size<n:
    size=min(size<<1,n);ag=conv(a[:size],g,size);ag += [0]*(size-len(ag));ag[0]=(2-ag[0])%MOD
    for i in range(1,size):ag[i]=(-ag[i])%MOD
    g=conv(g,ag,size);g += [0]*(size-len(g))
    return g[:n]
def derivative(a):return [i*a[i]%MOD for i in range(1,len(a))]
def integral(a):
    n=len(a)+1;inv=[0]*n
    if n>1:inv[1]=1
    for i in range(2,n):inv[i]=(MOD-(MOD//i)*inv[MOD%i])%MOD
    return [0]+[a[i-1]*inv[i]%MOD for i in range(1,n)]
def fps_log(a,n):
    r=integral(conv(derivative(a),fps_inv(a,n),n-1));return (r+[0]*n)[:n]
def fps_exp(a,n):
    g=[1];size=1
    while size<n:
        size=min(size<<1,n);lg=fps_log(g,size);delta=(a[:size]+[0]*size)[:size]
        for i in range(size):delta[i]=(delta[i]-lg[i])%MOD
        delta[0]=(delta[0]+1)%MOD;g=conv(g,delta,size);g += [0]*(size-len(g))
    return g[:n]
def fps_sqrt(a,n):
    g=[1];size=1;half=(MOD+1)//2
    while size<n:
        size=min(size<<1,n);q=conv(a[:size],fps_inv(g,size),size);q += [0]*(size-len(q));g += [0]*(size-len(q))
        g=[(g[i]+q[i])*half%MOD for i in range(size)]
    return g[:n]
def poly_mod(a,b):
    while a and a[-1]==0:a.pop()
    while b and b[-1]==0:b.pop()
    if len(a)<len(b):return a
    qn=len(a)-len(b)+1;q=conv(a[:qn],fps_inv(b[:qn],qn),qn)[:qn]
    prod=conv(q,b,len(a));r=[(a[i]-(prod[i] if i<len(prod) else 0))%MOD for i in range(len(b)-1)]
    while r and r[-1]==0:r.pop()
    return r
def product_tree(xs):
    size=1
    while size<len(xs):size<<=1
    tree=[[1] for _ in range(2*size)]
    for i,x in enumerate(xs):tree[size+i]=((-x)%MOD,1]
    for i in range(size-1,0,-1):tree[i]=conv(tree[i<<1],tree[i<<1|1])
    return tree,size
def evaluate(a,xs):
    if not xs:return []
    tree,size=product_tree(xs);ans=[0]*len(xs);stack=[(1,poly_mod(a[:],tree[1]))]
    while stack:
        v,r=stack.pop()
        if v>=size:
            i=v-size
            if i<len(xs):ans[i]=r[0] if r else 0
        else:
            stack.append((v<<1,poly_mod(r[:],tree[v<<1])));stack.append((v<<1|1,poly_mod(r[:],tree[v<<1|1]))

```

```

return ans
def interpolate(xs,ys):
    tree,size=product_tree(xs);values=evaluate(derivative(tree[1]),xs);work=[[0] for _ in range(2*size)
    for i in range(len(xs)):work[size+i]=[ys[i]*pow(values[i],MOD-2,MOD)%MOD]
    for v in range(size-1,0,-1):
        work[v]=conv(work[v<<1],tree[v<<1|1]);other=conv(work[v<<1|1],tree[v<<1])
        if len(work[v])<len(other):work[v]+=[0]*(len(other)-len(work[v]))
        for i,x in enumerate(other):work[v][i]=(work[v][i]+x)%MOD
    return (work[1]+[0]*len(xs))[:len(xs)]
def fwt(a,kind,invert=False):
    n=len(a);length=1
    while length<n:
        for mask in range(n):
            if mask&length:
                if kind=='or':a[mask]=(a[mask]-a[mask^length])%MOD if invert else (a[mask]+a[mask^length])%MOD
            else:
                u=a[mask^length];v=a[mask];a[mask^length]=(u+v)%MOD;a[mask]=(u-v)%MOD
        length<=<1
    if kind=='xor' and invert:
        z=pow(n,MOD-2,MOD)
        for i in range(n):a[i]=a[i]*z%MOD
def subset_conv(f,g,k):
    n=1<<k;F=[[0]*n for _ in range(k+1)];G=[[0]*n for _ in range(k+1)]
    for s in range(n):c=s.bit_count();F[c][s]=f[s];G[c][s]=g[s]
    for bit in range(k):
        step=1<<bit
        for s in range(n):
            if s&step:
                t=s^step
                for c in range(k+1):F[c][s]=(F[c][s]+F[c][t])%MOD;G[c][s]=(G[c][s]+G[c][t])%MOD
    H=[[0]*n for _ in range(k+1)]
    for s in range(n):
        for c in range(k+1):H[c][s]=sum(F[i][s]*G[c-i][s] for i in range(c+1))%MOD
    for bit in range(k):
        step=1<<bit
        for s in range(n):
            if s&step:
                t=s^step
                for c in range(k+1):H[c][s]=(H[c][s]-H[c][t])%MOD
    return [H[s.bit_count()][s] for s in range(n)]
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n,q=d[:2];a=d[2:2+n];xs=d[2+n:2+n+q];print(*evaluate
if __name__=='__main__':main()

```

B. Polynomial Interpolation

How to find it

Evaluate the derivative of the root product, form Lagrange weights, and merge them upward.

Correctness

The root expansion is the Lagrange formula. It matches every sample value and has degree below n , so uniqueness proves equality.

Complexity

$O(M(n) \log n)$ time.

The points must be distinct so that every $M'(X_i)$ is invertible. Preserve the association between each point and ordinate even if the tree is padded or balanced internally.

C++

```
#include <bits/stdc++.h>
using namespace std; using ll=long long;
const int MOD=998244353,G=3;
int power(int a,int e){ll r=1;for(;e;e>>=1,a=(ll)a*a%MOD)if(e&1)r=r*a%MOD;return r;}
void ntt(vector<int>&a,bool inv){int n=a.size();for(int i=1,j=0;i<n;i++){int b=n>>1;for(;j&b;b>>=1)
vector<int> conv(vector<int>a,vector<int>b,int need=-1){if(a.empty()||b.empty())return{};int total=
vector<int> inverse(const vector<int>&a,int n){vector<int>r(1,power(a[0],MOD-2));for(int sz=2;sz/2<
vector<int> derivative(const vector<int>&a){vector<int>d(max(0,(int)a.size()-1));for(int i=1;i<(int)
vector<int> integrate(const vector<int>&a){vector<int>r(a.size()+1),iv(a.size()+1);if(a.size())iv[1]
vector<int> logarithm(const vector<int>&a,int n){auto r=integrate(conv(derivative(a),inverse(a,n),n
vector<int> exponential(const vector<int>&a,int n){vector<int>r(1,1);for(int sz=2;sz/2<n;sz<=1){in
vector<int> square_root(const vector<int>&a,int n){vector<int>r(1,1);int half=(MOD+1)/2;for(int sz=
void trim(vector<int>&a){while(!a.empty()&&a.back()==0)a.pop_back();}
vector<int> poly_mod(vector<int>a,vector<int>b){trim(a);trim(b);if(a.size()<b.size())return a;vector
struct ProductTree{int n,size;vector<int>x;vector<vector<int>>t;ProductTree(vector<int>xs):n(xs.size
vector<int> eval(const vector<int>&a){vector<int>ans(n);function<void(int,vector<int>>>go=[&](int
vector<int> interp(const vector<int>&y){auto val=eval(derivative(t[1]));vector<vector<int>>w(2*size
void fwt(vector<int>&a,string kind,bool inv=false){int n=a.size();for(int bit=1;bit<n;bit<=1)for(i
vector<int> subset_conv(const vector<int>&f,const vector<int>&g,int k){int n=1<<k;vector<vector<int>
void print(const vector<int>&a){for(int i=0;i<(int)a.size();i++)cout<<a[i]<<" \n"[i+1==(int)a.size(
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;cin>>n;vector<int>x(n),y(n);for(int i
```

Python

```
import sys
MOD=998244353
ROOT=3
def ntt(a,invert):
    n=len(a);j=0
    for i in range(1,n):
        bit=n>>1
        while j&bit:j^=bit;bit>>=1
        j^=bit
        if i<j:a[i],a[j]=a[j],a[i]
```

```

length=2
while length<=n:
    wlen=pow(ROOT,(MOD-1)//length,MOD)
    if invert:wlen=pow(wlen,MOD-2,MOD)
    half=length>>1
    for i in range(0,n,length):
        w=1
        for j in range(i,i+half):
            u=a[j];v=a[j+half]*w%MOD
            a[j]=(u+v)%MOD;a[j+half]=(u-v)%MOD;w=w*wlen%MOD
        length<=1
    if invert:
        z=pow(n,MOD-2,MOD)
        for i in range(n):a[i]=a[i]*z%MOD
def conv(a,b,need=None):
    if not a or not b:return []
    total=len(a)+len(b)-1
    if min(len(a),len(b))<32:
        c=[0]*total
        for i,x in enumerate(a):
            for j,y in enumerate(b):c[i+j]=(c[i+j]+x*y)%MOD
    else:
        size=1
        while size<total:size<=1
        x=a+[0]*(size-len(a));y=b+[0]*(size-len(b));ntt(x,False);ntt(y,False)
        for i in range(size):x[i]=x[i]*y[i]%MOD
        ntt(x,True);c=x[:total]
    return c if need is None else c[:need]
def fps_inv(a,n):
    g=[pow(a[0],MOD-2,MOD)];size=1
    while size<n:
        size=min(size<<1,n);ag=conv(a[:size],g,size);ag += [0]*(size-len(ag));ag[0]=(2-ag[0])%MOD
        for i in range(1,size):ag[i]=(-ag[i])%MOD
        g=conv(g,ag,size);g += [0]*(size-len(g))
    return g[:n]
def derivative(a):return [i*a[i]%MOD for i in range(1,len(a))]
def integral(a):
    n=len(a)+1;inv=[0]*n
    if n>1:inv[1]=1
    for i in range(2,n):inv[i]=(MOD-(MOD//i)*inv[MOD%i])%MOD
    return [0]+[a[i-1]*inv[i]%MOD for i in range(1,n)]
def fps_log(a,n):
    r=integral(conv(derivative(a),fps_inv(a,n),n-1));return (r+[0]*n)[:n]
def fps_exp(a,n):
    g=[1];size=1
    while size<n:
        size=min(size<<1,n);lg=fps_log(g,size);delta=(a[:size]+[0]*size)[:size]
        for i in range(size):delta[i]=(delta[i]-lg[i])%MOD
        delta[0]=(delta[0]+1)%MOD;g=conv(g,delta,size);g += [0]*(size-len(g))
    return g[:n]
def fps_sqrt(a,n):

```

```

g=[1];size=1;half=(MOD+1)//2
while size<n:
    size=min(size<<1,n);q=conv(a[:size],fps_inv(g,size),size);q += [0]*(size-len(q));g += [0]*(size-1)
    g=[(g[i]+q[i])*half%MOD for i in range(size)]
    return g[:n]
def poly_mod(a,b):
    while a and a[-1]==0:a.pop()
    while b and b[-1]==0:b.pop()
    if len(a)<len(b):return a
    qn=len(a)-len(b)+1;q=conv(a[::-1][:qn],fps_inv(b[::-1],qn),qn)[::-1]
    prod=conv(q,b,len(a));r=[(a[i]-(prod[i] if i<len(prod) else 0))%MOD for i in range(len(b)-1)]
    while r and r[-1]==0:r.pop()
    return r
def product_tree(xs):
    size=1
    while size<len(xs):size<<=1
    tree=[[1] for _ in range(2*size)]
    for i,x in enumerate(xs):tree[size+i]=[(-x)%MOD,1]
    for i in range(size-1,0,-1):tree[i]=conv(tree[i<<1],tree[i<<1|1])
    return tree,size
def evaluate(a,xs):
    if not xs:return []
    tree,size=product_tree(xs);ans=[0]*len(xs);stack=[(1,poly_mod(a[:],tree[1]))]
    while stack:
        v,r=stack.pop()
        if v>=size:
            i=v-size
            if i<len(xs):ans[i]=r[0] if r else 0
        else:
            stack.append((v<<1,poly_mod(r[:],tree[v<<1])));stack.append((v<<1|1,poly_mod(r[:],tree[v<<1|1])))
    return ans
def interpolate(xs,ys):
    tree,size=product_tree(xs);values=evaluate(derivative(tree[1]),xs);work=[[0] for _ in range(2*size)]
    for i in range(len(xs)):work[size+i]=[ys[i]*pow(values[i],MOD-2,MOD)%MOD]
    for v in range(size-1,0,-1):
        work[v]=conv(work[v<<1],tree[v<<1|1]);other=conv(work[v<<1|1],tree[v<<1])
        if len(work[v])<len(other):work[v]+=[0]*(len(other)-len(work[v]))
        for i,x in enumerate(other):work[v][i]=(work[v][i]+x)%MOD
    return (work[1]+[0]*len(xs))[:len(xs)]
def fwt(a,kind,invert=False):
    n=len(a);length=1
    while length<n:
        for mask in range(n):
            if mask&length:
                if kind=='or':a[mask]=(a[mask]-a[mask^length])%MOD if invert else (a[mask]+a[mask^length])%MOD
            else:
                u=a[mask^length];v=a[mask];a[mask^length]=(u+v)%MOD;a[mask]=(u-v)%MOD
        length<<=1
    if kind=='xor' and invert:
        z=pow(n,MOD-2,MOD)
        for i in range(n):a[i]=a[i]*z%MOD

```

```

def subset_conv(f,g,k):
    n=1<<k;F=[[0]*n for _ in range(k+1)];G=[[0]*n for _ in range(k+1)]
    for s in range(n):c=s.bit_count();F[c][s]=f[s];G[c][s]=g[s]
    for bit in range(k):
        step=1<<bit
        for s in range(n):
            if s&step:
                t=s^step
                for c in range(k+1):F[c][s]=(F[c][s]+F[c][t])%MOD;G[c][s]=(G[c][s]+G[c][t])%MOD
    H=[[0]*n for _ in range(k+1)]
    for s in range(n):
        for c in range(k+1):H[c][s]=sum(F[i][s]*G[c-i][s] for i in range(c+1))%MOD
    for bit in range(k):
        step=1<<bit
        for s in range(n):
            if s&step:
                t=s^step
                for c in range(k+1):H[c][s]=(H[c][s]-H[c][t])%MOD
    return [H[s.bit_count()][s] for s in range(n)]
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n=d[0];xs=d[1::2];ys=d[2::2];print(*interpolate(x,ys))
if __name__=='__main__':main()

```

C. XOR Convolution

How to find it

Use Walsh–Hadamard butterflies because XOR is addition in the vector space of bits.

Correctness

Each Walsh character turns XOR into multiplication; inversion restores coefficients after pointwise products.

Complexity

$O(k \cdot 2^k)$ time and $O(2^k)$ storage.

The forward and inverse butterflies have the same additions and subtractions; only the inverse multiplies all entries by $\text{inverse}(2^k)$. A direct double loop over pairs of masks is the ideal small oracle.

C++

```

#include <bits/stdc++.h>
using namespace std; using ll=long long;
const int MOD=998244353,G=3;
int power(int a,int e){ll r=1;for(;e>=1;a=(ll)a*a%MOD)if(e&1)r=r*a%MOD;return r;}
void ntt(vector<int>&a,bool inv){int n=a.size();for(int i=1,j=0;i<n;i++){int b=n>>1;for(;j&b;b>=1)

```

```

vector<int> conv(vector<int>a,vector<int>b,int need=-1){if(a.empty()||b.empty())return{};int total=
vector<int> inverse(const vector<int>&a,int n){vector<int>r(1,power(a[0],MOD-2));for(int sz=2;sz/2<
vector<int> derivative(const vector<int>&a){vector<int>d(max(0,(int)a.size()-1));for(int i=1;i<(int)
vector<int> integrate(const vector<int>&a){vector<int>r(a.size()+1,iv(a.size()+1));if(a.size())iv[1]
vector<int> logarithm(const vector<int>&a,int n){auto r=integrate(conv(derivative(a),inverse(a,n),n
vector<int> exponential(const vector<int>&a,int n){vector<int>r(1,1);for(int sz=2;sz/2<n;sz<=1){in
vector<int> square_root(const vector<int>&a,int n){vector<int>r(1,1);int half=(MOD+1)/2;for(int sz
void trim(vector<int>&a){while(!a.empty()&&a.back()==0)a.pop_back();}
vector<int> poly_mod(vector<int>a,vector<int>b){trim(a);trim(b);if(a.size()<b.size())return a;vector
struct ProductTree{int n,size;vector<int>x;vector<vector<int>>t;ProductTree(vector<int>xs):n(xs.size
vector<int> eval(const vector<int>&a){vector<int>ans(n);function<void(int,vector<int>>>go=[&](int
vector<int> interp(const vector<int>&y){auto val=eval(derivative(t[1]));vector<vector<int>>w(2*size
void fwt(vector<int>&a,string kind,bool inv=false){int n=a.size();for(int bit=1;bit<n;bit<=1)for(i
vector<int> subset_conv(const vector<int>&f,const vector<int>&g,int k){int n=1<<k;vector<vector<int>
void print(const vector<int>&a){for(int i=0;i<(int)a.size();i++)cout<<a[i]<<" \n"[i+1==(int)a.size(
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int k;cin>>k;int n=1<<k;vector<int>a(n),b(n

```

Python

```

import sys
MOD=998244353
ROOT=3
def ntt(a,invert):
    n=len(a);j=0
    for i in range(1,n):
        bit=n>>1
        while j&bit:j^=bit;bit>>=1
        j^=bit
        if i<j:a[i],a[j]=a[j],a[i]
    length=2
    while length<=n:
        wlen=pow(ROOT,(MOD-1)//length,MOD)
        if invert:wlen=pow(wlen,MOD-2,MOD)
        half=length>>1
        for i in range(0,n,length):
            w=1
            for j in range(i,i+half):
                u=a[j];v=a[j+half]*w%MOD
                a[j]=(u+v)%MOD;a[j+half]=(u-v)%MOD;w=w*wlen%MOD
            length<=1
    if invert:
        z=pow(n,MOD-2,MOD)
        for i in range(n):a[i]=a[i]*z%MOD
def conv(a,b,need=None):
    if not a or not b:return []
    total=len(a)+len(b)-1
    if min(len(a),len(b))<32:
        c=[0]*total
        for i,x in enumerate(a):

```

```

    for j,y in enumerate(b):c[i+j]=(c[i+j]+x*y)%MOD
else:
    size=1
    while size<total:size<=1
    x=a+[0]*(size-len(a));y=b+[0]*(size-len(b));ntt(x,False);ntt(y,False)
    for i in range(size):x[i]=x[i]*y[i]%MOD
    ntt(x,True);c=x[:total]
    return c if need is None else c[:need]
def fps_inv(a,n):
    g=[pow(a[0],MOD-2,MOD)];size=1
    while size<n:
        size=min(size<<1,n);ag=conv(a[:size],g,size);ag += [0]*(size-len(ag));ag[0]=(2-ag[0])%MOD
        for i in range(1,size):ag[i]=(-ag[i])%MOD
        g=conv(g,ag,size);g += [0]*(size-len(g))
    return g[:n]
def derivative(a):return [i*a[i]%MOD for i in range(1,len(a))]
def integral(a):
    n=len(a)+1;inv=[0]*n
    if n>1:inv[1]=1
    for i in range(2,n):inv[i]=(MOD-(MOD//i)*inv[MOD%i])%MOD
    return [0]+[a[i-1]*inv[i]%MOD for i in range(1,n)]
def fps_log(a,n):
    r=integral(conv(derivative(a),fps_inv(a,n),n-1));return (r+[0]*n)[:n]
def fps_exp(a,n):
    g=[1];size=1
    while size<n:
        size=min(size<<1,n);lg=fps_log(g,size);delta=(a[:size]+[0]*size)[:size]
        for i in range(size):delta[i]=(delta[i]-lg[i])%MOD
        delta[0]=(delta[0]+1)%MOD;g=conv(g,delta,size);g += [0]*(size-len(g))
    return g[:n]
def fps_sqrt(a,n):
    g=[1];size=1;half=(MOD+1)//2
    while size<n:
        size=min(size<<1,n);q=conv(a[:size],fps_inv(g,size),size);q += [0]*(size-len(q));g += [0]*(size-1)
        g=[(g[i]+q[i])*half%MOD for i in range(size)]
    return g[:n]
def poly_mod(a,b):
    while a and a[-1]==0:a.pop()
    while b and b[-1]==0:b.pop()
    if len(a)<len(b):return a
    qn=len(a)-len(b)+1;q=conv(a[:qn],fps_inv(b[:qn],qn),qn)[:qn]
    prod=conv(q,b,len(a));r=[(a[i]-(prod[i] if i<len(prod) else 0))%MOD for i in range(len(b)-1)]
    while r and r[-1]==0:r.pop()
    return r
def product_tree(xs):
    size=1
    while size<len(xs):size<=1
    tree=[[1] for _ in range(2*size)]
    for i,x in enumerate(xs):tree[size+i]=[(-x)%MOD,1]
    for i in range(size-1,0,-1):tree[i]=conv(tree[i<<1],tree[i<<1|1])
    return tree,size

```

```

def evaluate(a,xs):
    if not xs: return []
    tree,size=product_tree(xs);ans=[0]*len(xs);stack=[(1,poly_mod(a[:],tree[1]))]
    while stack:
        v,r=stack.pop()
        if v>=size:
            i=v-size
            if i<len(xs):ans[i]=r[0] if r else 0
        else:
            stack.append((v<<1,poly_mod(r[:],tree[v<<1])));stack.append((v<<1|1,poly_mod(r[:],tree[v<<1|1])))
    return ans

def interpolate(xs,ys):
    tree,size=product_tree(xs);values=evaluate(derivative(tree[1]),xs);work=[[0] for _ in range(2*size)]
    for i in range(len(xs)):work[size+i]=[ys[i]*pow(values[i],MOD-2,MOD)%MOD]
    for v in range(size-1,0,-1):
        work[v]=conv(work[v<<1],tree[v<<1|1]);other=conv(work[v<<1|1],tree[v<<1])
        if len(work[v])<len(other):work[v]+=[0]*(len(other)-len(work[v]))
        for i,x in enumerate(other):work[v][i]=(work[v][i]+x)%MOD
    return (work[1]+[0]*len(xs))[:len(xs)]

def fwt(a,kind,invert=False):
    n=len(a);length=1
    while length<n:
        for mask in range(n):
            if mask&length:
                if kind=='or':a[mask]=(a[mask]-a[mask^length])%MOD if invert else (a[mask]+a[mask^length])%MOD
            else:
                u=a[mask^length];v=a[mask];a[mask^length]=(u+v)%MOD;a[mask]=(u-v)%MOD
        length<=1
    if kind=='xor' and invert:
        z=pow(n,MOD-2,MOD)
        for i in range(n):a[i]=a[i]*z%MOD

def subset_conv(f,g,k):
    n=1<<k;F=[[0]*n for _ in range(k+1)];G=[[0]*n for _ in range(k+1)]
    for s in range(n):c=s.bit_count();F[c][s]=f[s];G[c][s]=g[s]
    for bit in range(k):
        step=1<<bit
        for s in range(n):
            if s&step:
                t=s^step
                for c in range(k+1):F[c][s]=(F[c][s]+F[c][t])%MOD;G[c][s]=(G[c][s]+G[c][t])%MOD
    H=[[0]*n for _ in range(k+1)]
    for s in range(n):
        for c in range(k+1):H[c][s]=sum(F[i][s]*G[c-i][s] for i in range(c+1))%MOD
    for bit in range(k):
        step=1<<bit
        for s in range(n):
            if s&step:
                t=s^step
                for c in range(k+1):H[c][s]=(H[c][s]-H[c][t])%MOD
    return [H[s.bit_count()][s] for s in range(n)]

def main():

```

```
d=list(map(int,sys.stdin.buffer.read().split()));k=d[0];n=1<<k;a=d[1:1+n];b=d[1+n:1+2*n];fwt(a,'xor')
if __name__=='__main__':main()
```

D. Subset Convolution

How to find it

Separate values by popcount, zeta-transform every rank, convolve ranks, then Möbius-invert.

Correctness

Rank equality removes overlapping pairs: if ranks add to $|S|$ and both sets lie in S , their union is S exactly when they are disjoint.

Complexity

$O(k^2 2^k)$ time and $O(k 2^k)$ storage.

After zeta transformation, convolve the popcount ranks at each mask. Möbius inversion must be applied rank by rank before selecting rank $|S|$; omitting the ranks computes a different convolution that includes overlapping pairs.

C++

```
#include <bits/stdc++.h>
using namespace std; using ll=long long;
const int MOD=998244353,G=3;
int power(int a,int e){ll r=1;for(;e>=1;a=(ll)a*a%MOD)if(e&1)r=r*a%MOD;return r;}
void ntt(vector<int>&a,bool inv){int n=a.size();for(int i=1,j=0;i<n;i++){int b=n>>1;for(;j<b;b>=1),
vector<int> conv(vector<int>a,vector<int>b,int need=-1){if(a.empty()||b.empty())return{};int total=
vector<int> inverse(const vector<int>&a,int n){vector<int>r(1,power(a[0],MOD-2));for(int sz=2;sz/2<
vector<int> derivative(const vector<int>&a){vector<int>d(max(0,(int)a.size()-1));for(int i=1;i<(int)
vector<int> integrate(const vector<int>&a){vector<int>r(a.size()+1),iv(a.size()+1);if(a.size())iv[1]
vector<int> logarithm(const vector<int>&a,int n){auto r=integrate(conv(derivative(a),inverse(a,n),n
vector<int> exponential(const vector<int>&a,int n){vector<int>r(1,1);for(int sz=2;sz/2<n;sz<=1){in
vector<int> square_root(const vector<int>&a,int n){vector<int>r(1,1);int half=(MOD+1)/2;for(int sz=
void trim(vector<int>&a){while(!a.empty()&&a.back()==0)a.pop_back();}
vector<int> poly_mod(vector<int>a,vector<int>b){trim(a);trim(b);if(a.size()<b.size())return a;vector
struct ProductTree{int n,size;vector<int>x;vector<vector<int>>t;ProductTree(vector<int>xs):n(xs.size
vector<int> eval(const vector<int>&a){vector<int>ans(n);function<void(int,vector<int>>>go=[&](int
vector<int> interp(const vector<int>&y){auto val=eval(derivative(t[1]));vector<vector<int>>w(2*size
void fwt(vector<int>&a,string kind,bool inv=false){int n=a.size();for(int bit=1;bit<n;bit<=1)for(i
vector<int> subset_conv(const vector<int>&f,const vector<int>&g,int k){int n=1<<k;vector<vector<int>
void print(const vector<int>&a){for(int i=0;i<(int)a.size();i++)cout<<a[i]<<" \n"[i+1==(int)a.size(
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int k;cin>>k;int n=1<<k;vector<int>a(n),b(n
```

Python

```
import sys
MOD=998244353
ROOT=3
def ntt(a,invert):
    n=len(a);j=0
    for i in range(1,n):
        bit=n>>1
        while j&bit:j^=bit;bit>>=1
        j^=bit
        if i<j:a[i],a[j]=a[j],a[i]
    length=2
    while length<=n:
        wlen=pow(ROOT,(MOD-1)//length,MOD)
        if invert:wlen=pow(wlen,MOD-2,MOD)
        half=length>>1
        for i in range(0,n,length):
            w=1
            for j in range(i,i+half):
                u=a[j];v=a[j+half]*w%MOD
                a[j]=(u+v)%MOD;a[j+half]=(u-v)%MOD;w=w*wlen%MOD
            length<=<=1
        if invert:
            z=pow(n,MOD-2,MOD)
            for i in range(n):a[i]=a[i]*z%MOD
def conv(a,b,need=None):
    if not a or not b:return []
    total=len(a)+len(b)-1
    if min(len(a),len(b))<32:
        c=[0]*total
        for i,x in enumerate(a):
            for j,y in enumerate(b):c[i+j]=(c[i+j]+x*y)%MOD
    else:
        size=1
        while size<total:size<=<=1
        x=a+[0]*(size-len(a));y=b+[0]*(size-len(b));ntt(x,False);ntt(y,False)
        for i in range(size):x[i]=x[i]*y[i]%MOD
        ntt(x,True);c=x[:total]
    return c if need is None else c[:need]
def fps_inv(a,n):
    g=[pow(a[0],MOD-2,MOD)];size=1
    while size<n:
        size=min(size<<1,n);ag=conv(a[:size],g,size);ag += [0]*(size-len(ag));ag[0]=(2-ag[0])%MOD
        for i in range(1,size):ag[i]=(-ag[i])%MOD
        g=conv(g,ag,size);g += [0]*(size-len(g))
    return g[:n]
def derivative(a):return [i*a[i]%MOD for i in range(1,len(a))]
def integral(a):
    n=len(a)+1;inv=[0]*n
    if n>1:inv[1]=1
```

```

for i in range(2,n):inv[i]=(MOD-(MOD//i)*inv[MOD%i])%MOD
return [0]+[a[i-1]*inv[i]%MOD for i in range(1,n)]
def fps_log(a,n):
    r=integral(conv(derivative(a),fps_inv(a,n),n-1));return (r+[0]*n)[:n]
def fps_exp(a,n):
    g=[1];size=1
    while size<n:
        size=min(size<<1,n);lg=fps_log(g,size);delta=(a[:size]+[0]*size)[:size]
        for i in range(size):delta[i]=(delta[i]-lg[i])%MOD
        delta[0]=(delta[0]+1)%MOD;g=conv(g,delta,size);g += [0]*(size-len(g))
    return g[:n]
def fps_sqrt(a,n):
    g=[1];size=1;half=(MOD+1)//2
    while size<n:
        size=min(size<<1,n);q=conv(a[:size],fps_inv(g,size),size);q += [0]*(size-len(q));g += [0]*(size-len(q))
        g=[(g[i]+q[i])*half%MOD for i in range(size)]
    return g[:n]
def poly_mod(a,b):
    while a and a[-1]==0:a.pop()
    while b and b[-1]==0:b.pop()
    if len(a)<len(b):return a
    qn=len(a)-len(b)+1;q=conv(a[:qn],fps_inv(b[:qn],qn),qn)[:qn]
    prod=conv(q,b,len(a));r=[(a[i]-(prod[i] if i<len(prod) else 0))%MOD for i in range(len(b)-1)]
    while r and r[-1]==0:r.pop()
    return r
def product_tree(xs):
    size=1
    while size<len(xs):size<<=1
    tree=[[1] for _ in range(2*size)]
    for i,x in enumerate(xs):tree[size+i]=[-x%MOD,1]
    for i in range(size-1,0,-1):tree[i]=conv(tree[i<<1],tree[i<<1|1])
    return tree,size
def evaluate(a,xs):
    if not xs:return []
    tree,size=product_tree(xs);ans=[0]*len(xs);stack=[(1,poly_mod(a[:],tree[1]))]
    while stack:
        v,r=stack.pop()
        if v>=size:
            i=v-size
            if i<len(xs):ans[i]=r[0] if r else 0
        else:
            stack.append((v<<1,poly_mod(r[:],tree[v<<1])));stack.append((v<<1|1,poly_mod(r[:],tree[v<<1|1])))
    return ans
def interpolate(xs,ys):
    tree,size=product_tree(xs);values=evaluate(derivative(tree[1]),xs);work=[[0] for _ in range(2*size)]
    for i in range(len(xs)):work[size+i]=[ys[i]*pow(values[i],MOD-2,MOD)%MOD]
    for v in range(size-1,0,-1):
        work[v]=conv(work[v<<1],tree[v<<1|1]);other=conv(work[v<<1|1],tree[v<<1])
        if len(work[v])<len(other):work[v]+=[0]*(len(other)-len(work[v]))
        for i,x in enumerate(other):work[v][i]=(work[v][i]+x)%MOD
    return (work[1]+[0]*len(xs))[:len(xs)]

```

```

def fwt(a,kind,invert=False):
    n=len(a);length=1
    while length<n:
        for mask in range(n):
            if mask&length:
                if kind=='or':a[mask]=(a[mask]-a[mask^length])%MOD if invert else (a[mask]+a[mask^length])%MOD
            else:
                u=a[mask^length];v=a[mask];a[mask^length]=(u+v)%MOD;a[mask]=(u-v)%MOD
        length<=1
    if kind=='xor' and invert:
        z=pow(n,MOD-2,MOD)
        for i in range(n):a[i]=a[i]*z%MOD
def subset_conv(f,g,k):
    n=1<<k;F=[[0]*n for _ in range(k+1)];G=[[0]*n for _ in range(k+1)]
    for s in range(n):c=s.bit_count();F[c][s]=f[s];G[c][s]=g[s]
    for bit in range(k):
        step=1<<bit
        for s in range(n):
            if s&step:
                t=s^step
                for c in range(k+1):F[c][s]=(F[c][s]+F[c][t])%MOD;G[c][s]=(G[c][s]+G[c][t])%MOD
    H=[[0]*n for _ in range(k+1)]
    for s in range(n):
        for c in range(k+1):H[c][s]=sum(F[i][s]*G[c-i][s] for i in range(c+1))%MOD
    for bit in range(k):
        step=1<<bit
        for s in range(n):
            if s&step:
                t=s^step
                for c in range(k+1):H[c][s]=(H[c][s]-H[c][t])%MOD
    return [H[s.bit_count()][s] for s in range(n)]
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));k=d[0];n=1<<k;f=d[1:1+n];g=d[1+n:1+2*n];print(*su
if __name__=='__main__':main()

```

Distinguishing the tools

Use a product tree when many unrelated scalar evaluation moduli share one polynomial. Use Walsh butterflies for XOR, subset-zeta for OR, supermask-zeta for AND, and ranked subset-zeta for disjoint union. Stress each implementation against its literal defining sum on dimensions up to five; this catches a reversed loop direction more reliably than large samples.